Java advanced



nexign <academy>

План курса

- SOLID
- Паттерны проектирования
- Http servlets
- web app (JDBC + servlet)
- Cuncurrency

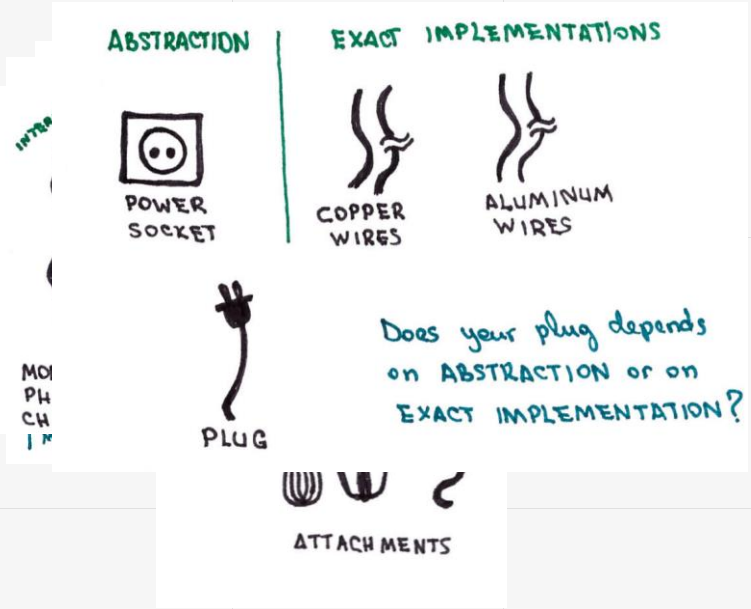
SOLID

* набор принципов озвученных Майклом Фэзерсом в начале 00х

Что такое SOLID?

Акроним SOLID расшифровывается как:

- **S: Single Responsibility Principle** (Принцип единственной ответственности).
- **O: Open-Closed Principle** (Принцип открытости-закрытости).
- **L: Liskov Substitution Principle** (Принцип подстановки Барбары Лисков).
- **I: Interface Segregation Principle** (Принцип разделения интерфейса).
- **D: Dependency Inversion Principle** (Принцип инверсии зависимостей).



Single Responcibility

Единая ответстственность

«У класса должна быть всего одна причина для изменения.»

«Каждый класс должен делать что-то одно.»



Нарушение:
Изменение одной части приложения ломает всё

Open - close principal

Принцип открытости - закрытости

«Сущности должны быть открыты для расширения, но закрыты для изменения.»

«То есть можно добавлять новый функционал без изменения существующего кода.»



Нарушение:
Добавление новой фичи требует переписывать старый код

Liskov Substitution Principle

Принцип подстановки Барбары Лисков

«Объекты подтипов должны быть заменяемы на объекты базового типа без изменения правильности программы.»

«Дочерний класс должен заменять родительский без изменений в поведении программы.»



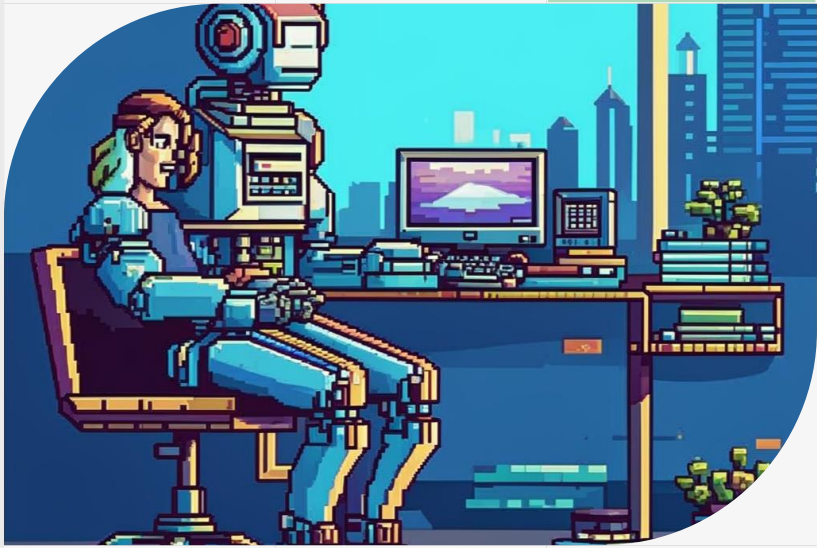
Нарушение:
Наследники ведут себя непредсказуемо

Interface Segregation

Принцип разделения интерфейсов

«Клиенты не должны зависеть от методов, которыми они не пользуются.»

«Большие интерфейсы — зло. Лучше несколько маленьких, чем один огромный.



Нарушение:
Перегруженные интерфейсы заставляют писать лишний код

Dependency Injection

Внедрение зависимостей

«Высокоуровневые модули не должны зависеть от низкоуровневых..»

«Всё должно зависеть от абстракций.»



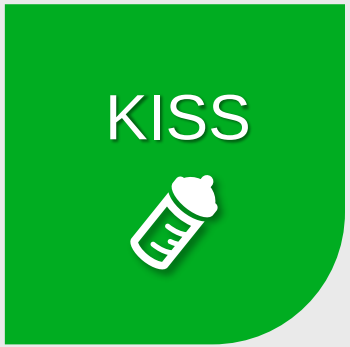
Нарушение:
Зависимость от конкретных реализаций делает код негибким

Еще принципы



Don’t repeat yourself

избегать дублирования кода



Keep it simple, stupid

Чем проще решение задачи, тем легче его понять, поддерживать и изменять



You ain’t gonna need it

следует реализовывать только те функции, которые действительно необходимы прямо сейчас

Практическое задание

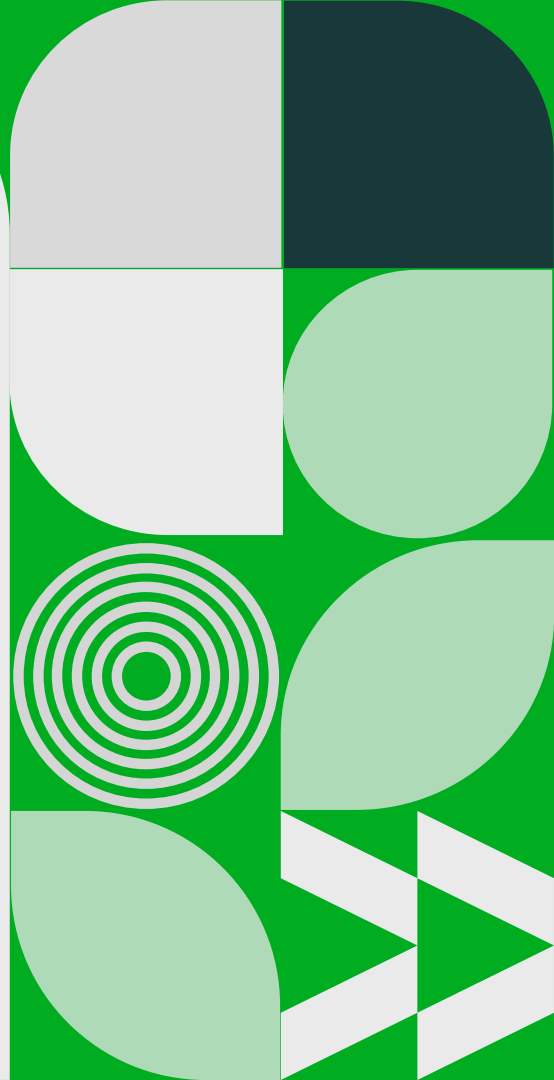
Отсмотреть код кофейни и найти 3 места, где можно отрефакторить код с учетом принципов SOLID

Паттерны проектирования

Паттерны проектирования — это решения распространенных проблем при разработке приложений. Также они известны как шаблоны проектирования, паттерны объектно-ориентированного программирования и design patterns. В отличие от готовых функций или библиотек, паттерн представляет собой не конкретный код, а общую концепцию решения проблемы, которую еще нужно подстроить под задачи.

Виды паттернов

- Порождающие → Singleton, Builder, Factory method, Prototype, Abstract factory …
- Структурные → Adapter, Decorator, Proxy, Composite, Façade …
- Поведенческие → Template method, Iterator, Observer, Chain of responsibility, Strategy …



nexign <academy>

https://refactoring.guru/

Порождающие

Singleton

Одиночка



Singleton гарантирует, что созданный объект будет единственным в приложении и позволяет получать к нему доступ из любой части приложения.

Пример: классный журнал в школе. У каждого класса он только один, и если учитель спросит журнал, то всегда получит один и тот же экземпляр.

Когда нужен: пригодится, если в приложении есть управляющий объект, в котором хранится весь контекст приложения. Например, в сервисах хранения данных.

Builder

Строитель

Builder позволяет создавать разные объекты с заданным состоянием, используя один и тот же код.

Пример: заказ пиццы. При выборе мы можем указать какую начинку мы хотим или не хотим. Таким образом, мы можем создавать различные виды одного объекта.

Когда нужен: пригодится при составлении SQL-запроса, а также в юнит-тестах.



Factory method

Фабрика

Паттерн позволяет использовать один класс для создания объектов разной реализации интерфейса и делегировать логику дочерним классам.

Проще: метод, который создаёт объект

Когда нужен: паттерн подходит для ситуаций, когда нам необходимо выполнить действие (например, отправить запрос), но заранее неизвестны все данные, так как они зависят от входных параметров (например, от протокола передачи данных — rest, soap или socket).

Abstract factory

Абстрактная фабрика

Abstract Factory позволяет использовать один интерфейс для создания семейств связанных объектов, не указывая их конкретных классов. Логика создания объектов делегируется дочерним классам (конкретным фабрикам), которые реализуют этот интерфейс.

Проще: Это фабрика фабрик. Она создаёт объекты, которые относятся к одному семейству.

Когда нужен: Паттерн подходит для ситуаций, когда нужно создавать группы связанных объектов, но заранее неизвестно, какие именно объекты потребуются, так как они зависят от контекста.

Структурные

Adapter

Adapter позволяет объектам с несовместимыми интерфейсами работать вместе.

Еще проще: европейские розетки отличаются от английских, поэтому, приезжая в Лондон, туристы обязательно берут переходник, или адаптер.

Когда нужен: часто используется, если мы работаем со сторонней библиотекой или компонентом, доступ к изменениями методов которого у нас отсутствует. Или когда есть несколько сторонних систем, доступ к которым должен осуществляться единообразно через одинаковый интерфейс.



Decorator

Паттерн позволяет добавлять объектам новые функции с помощью обертки без создания отдельного класса.

Пример: подключение мышки к ноутбуку, то есть вы добавляете новую функцию устройству, не меняя его.

Когда нужен: у этого паттерна широкая область применения. Им пользуются каждый раз, когда нужно добавить логику в уже созданные объекты и библиотеки, менять которые нельзя.



Facade

Facade предоставляет упрощенный интерфейс для сложной системы.

Пример: При оплате покупки через Pay-сервисы вы подносите телефон к устройству и оплачиваете покупку. Кажется, что все просто. Но на самом деле внутри этого процесса происходит гораздо больше вещей. Этот упрощенный интерфейс называется фасадом.

Когда нужен: Используется в библиотеках и позволяет описать их так, чтобы пользователю не нужно было вникать в их реализацию.



Proxy

Заместитель

предоставляет объект-заменитель, который контролирует доступ к другому объекту. Заместитель может выполнять дополнительные действия до или после обращения к реальному объекту, такие как ленивая инициализация, контроль доступа, логирование или кэширование.

Когда нужен:

- Когда нужно контролировать доступ к объекту (например, проверка прав доступа).
- Когда нужно лениво инициализировать объект (создавать его только при необходимости).
- Когда нужно добавлять дополнительную логику (например, логирование или кэширование) без изменения реального объекта.



Поведенческие

Mediator

Посредник

позволяет уменьшить связанность множества классов между собой, перенося взаимодействие между ними в отдельный класс-посредник. Вместо того чтобы объекты напрямую общались друг с другом, они взаимодействуют через посредника, который управляет их взаимодействием.

Проще: Это как диспетчер, который управляет общением между объектами, чтобы они не зависели друг от друга напрямую.

Когда нужен:

- Когда у вас есть множество объектов, которые взаимодействуют друг с другом, и их связи становятся слишком сложными.
- Когда нужно централизовать управление взаимодействием между объектами.
- Когда нужно уменьшить связанность между классами.



Observer

Наблюдатель

Observer позволяет одному объекту подписываться на другие объекты и отслеживать их изменения

Еще проще: после того как вы оформили подписку на газету или журнал, вам больше не нужно ездить в супермаркет и проверять, вышел ли очередной номер. Издательство будет присылать новые номера по почте прямо к вам домой сразу после их выхода.

Когда нужен: часто используется при проектировании библиотек, которые управляют состоянием приложения. Также является основой для различных классов, способных реагировать на изменения и уведомлять своих подписчиков об этих изменениях



Strategy

Стратегия

Позволяет определить семейство алгоритмов, инкапсулировать каждый из них и делать их взаимозаменяемыми. Это позволяет изменять алгоритмы независимо от клиента, который их использует.

Проще: Это как выбор разных способов выполнения одной и той же задачи. Например, выбор способа оплаты (картой, наличными, криптовалютой) в зависимости от ситуации.

Когда нужен:

- Когда нужно использовать разные варианты одного и того же алгоритма.
- Когда есть множество похожих классов, отличающихся только поведением.
- Когда нужно изолировать код алгоритмов от клиентского кода.



Практическое задание

Посмотреть код кофейни и придумать 3 способа использования паттернов

Java advanced
День 2



nexign <academy>

Проблема

```
public record User(Long id, String login) {

    @Override
    public String toString() {
        return "Пользователь: " +
                id + " " + login;
    }
}

public class DumbUserService {
    private final Map<Long, User> storage =   new HashMap<>();
}
```

- При перезапуске данные стираются
- Продолжительная работа приведет к падению по памяти

Решение: использовать БД

Реляционные БД

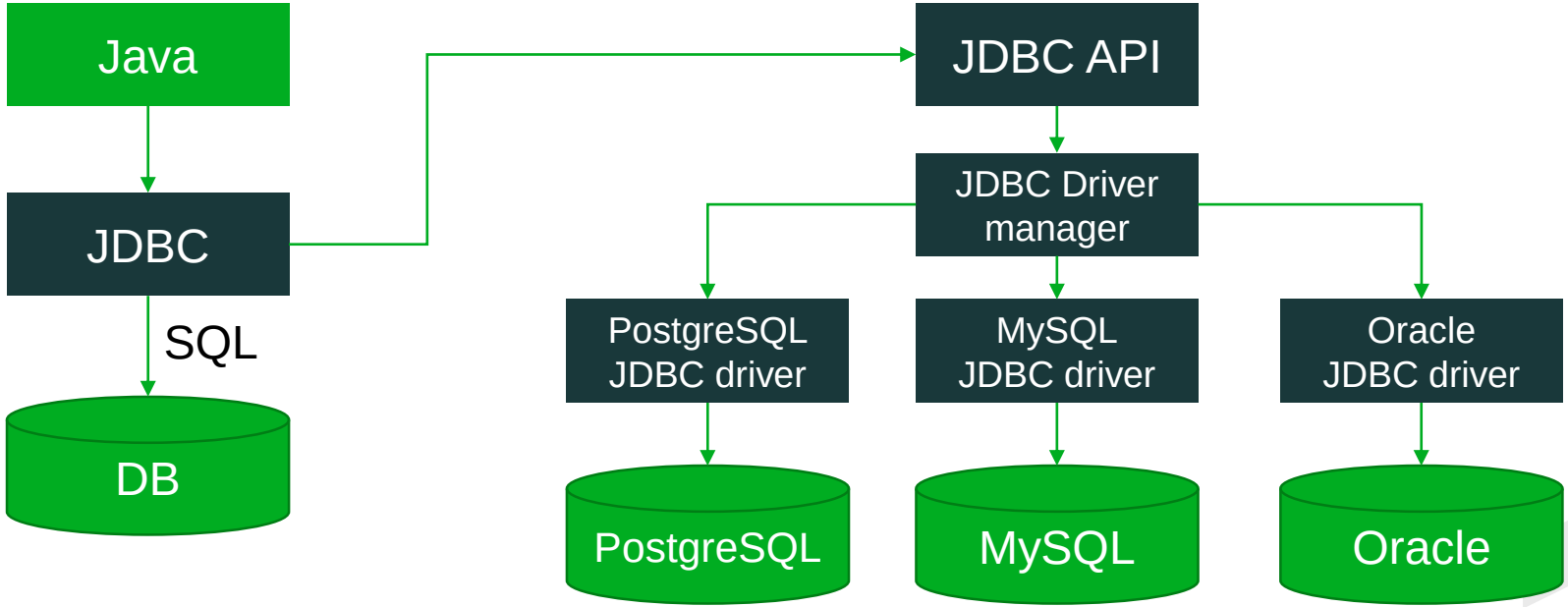
Хранят данные в таблицах

id	login
1	Alex
2	Stas

orderId	orderName	coast
3	Бумага	0.10
4	Ручка	0.25

id	orderId	timeStamp
1	3	12:00:00
2	4	18:05:05

Java database connectivity (JDBC)



Уровни изоляции транзакций

- **Connection.TRANSACTION_NONE (0):** Транзакции не поддерживаются.
- **Connection.TRANSACTION_READ_UNCOMMITTED (1):** Поддерживает "грязные чтения".
- **Connection.TRANSACTION_READ_COMMITTED (2)*:** Поддерживает "неповторяемые чтения".
- **Connection.TRANSACTION_REPEATABLE_READ (4):** Поддерживает "повторяемые чтения«
- **Connection.TRANSACTION_SERIALIZABLE (8):** Самый строгий уровень изоляции. Он предотвращает все типы аномалий

* - по умолчанию

DDL и DML

- **DDL (Data Definition Language) -** язык определения данных, используется для создания, изменения и удаления объектов базы данных (таблиц, индексов, представлений и т.д.).

  - CREATE TABLE, CREATE INDEX, CREATE VIEW - создание объектов
  - ALTER TABLE, ALTER INDEX - изменение объектов
  - DROP TABLE, DROP INDEX, DROP VIEW - удаление объектов

DML (Data Manipulation Language) - язык манипулирования данными, используется для вставки, обновления, удаления и выборки данных из таблиц.

- SELECT - выборка данных
- INSERT - вставка новых данных
- UPDATE - обновление существующих данных
- DELETE - удаление данных

Statement implements Autoclosable

- это интерфейс в Java, который используется для отправки SQL-запросов в базу данных и получения результатов.

Методы:

- **execute(sql)** - универсальный запрос
- **int executeUpdate(sql)** – INSERT, UPDATE, DELETE. Вернет кол-во измененных записей
- **ResultSet<> executeQuery()** – select запросы

```
try (Connection connection = DriverManager.getConnection(URL_KEY, USER_KEY, PASSWORD_KEY);
    var statemant = connection.createStatement(sql)) {
    System.out.println(statemant.execute(sql));
```

PrepareStetement

Плюсы:

- Проверяет запрос перед отправкой, что добавляет безопасности
- Передача параметров через ?
- SQL передается в самом стейтменте а не в методах

```
String sql = """
        select orderName
        from orders
        where orderId = ?
        """;

try (Connection connection = DriverManager.getConnection(URL_KEY, USER_KEY, PASSWORD_KEY);
     PreparedStatement statement = connection.prepareStatement(sql)) {
    statement.setLong(1, orderId);
```

ResultSet



id	Login
1	Alex
2	Stas
3	Misha

- Поле можно определить по порядковому номеру или по имени
- Закрывается с объектом statement

```
while (resultSet.next()) {
        str = resultSet.getString(“Login");
}
```

Statement

- **setFetchSize(int size)** – для указания количества строк, которые должны извлекаться из базы данных за один раз при выполнении запроса
- **setQueryTimeout(int t) -** метод указывает максимальное время выполнения запроса в секундах. Если запрос не завершится в указанный срок, будет выброшено исключение SQLException.
- **setMaxRows(int max) -** ограничения на максимальное количество строк, которые может содержать объект ResultSet. Если количество строк, возвращаемых запросом, превышает указанное значение, лишние строки просто игнорируются.

Транзакция

Единица работы в рамках соединения с БД
По умолчанию установлен

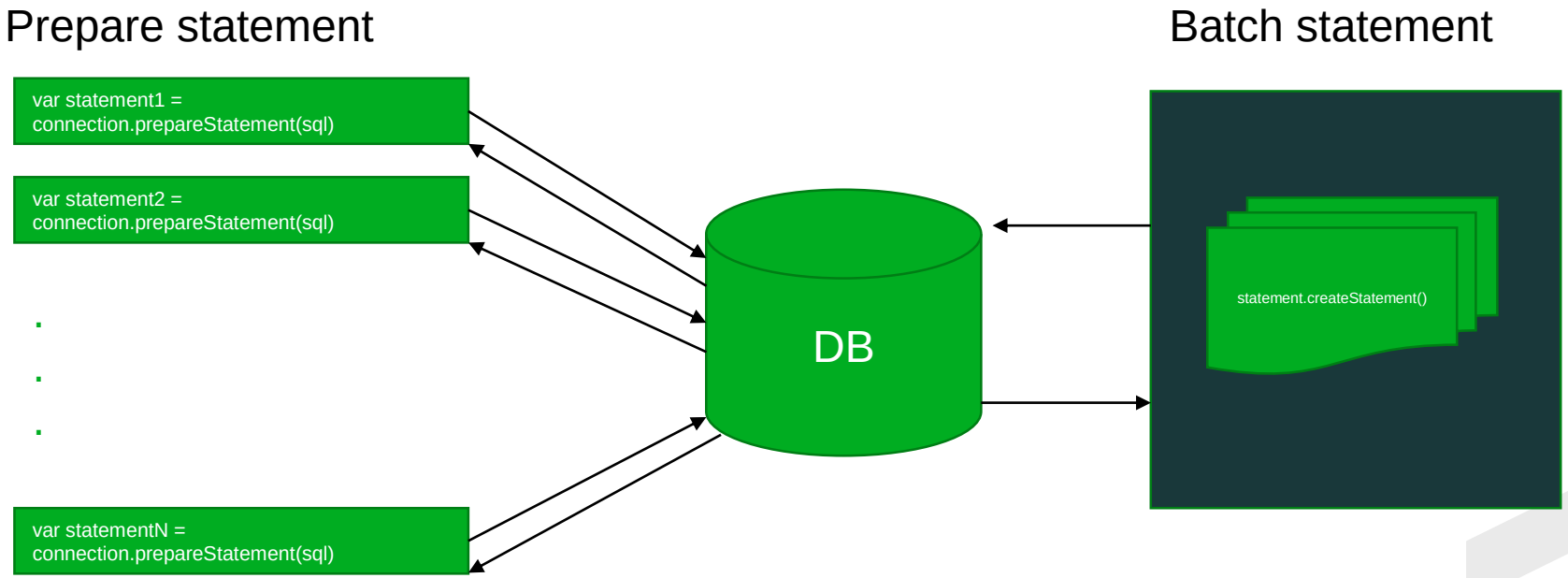
connection.setAutoCommited(false);

Ручное управление:
- connection.commit()
- connection.rollback()

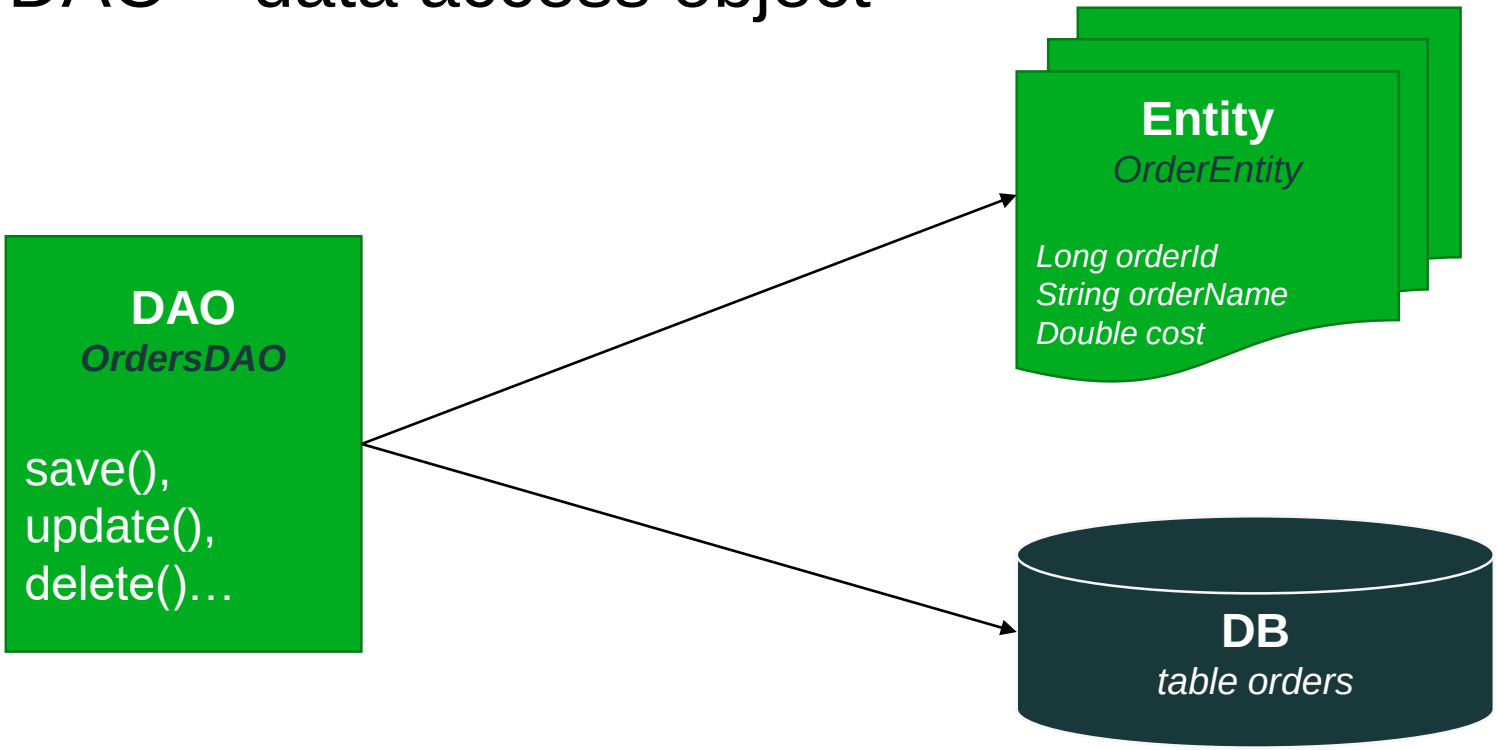
commit

rollback

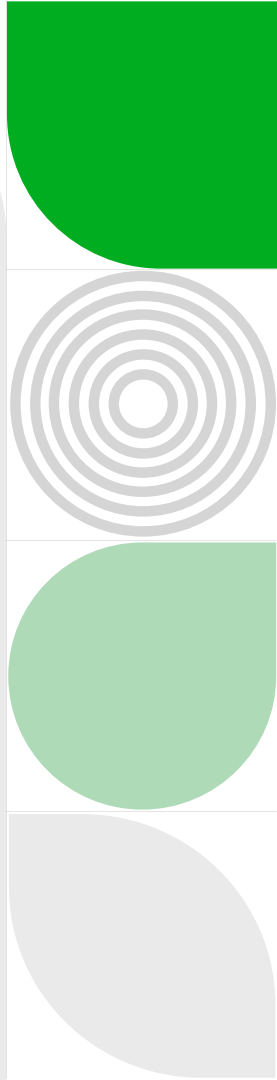
Batch операции



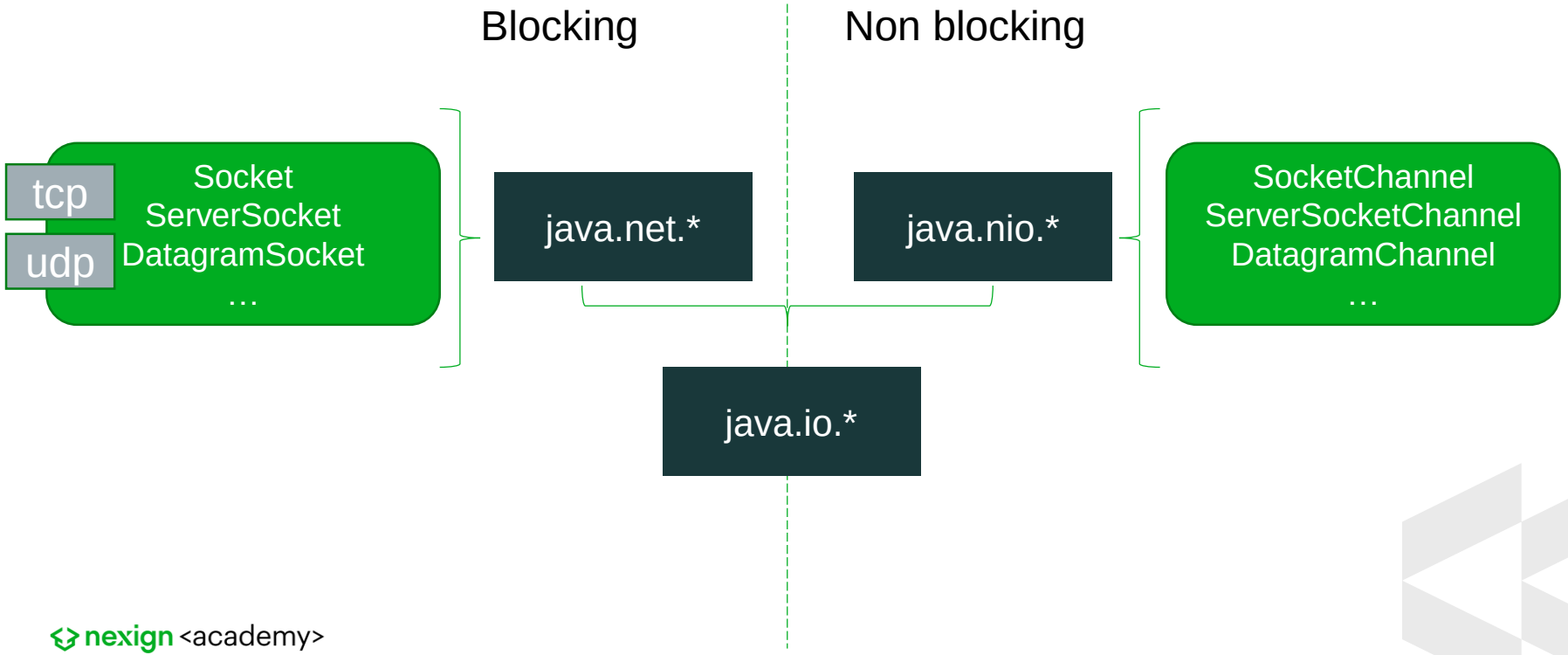
DAO – data access object



java.net.*
Работа с сетью

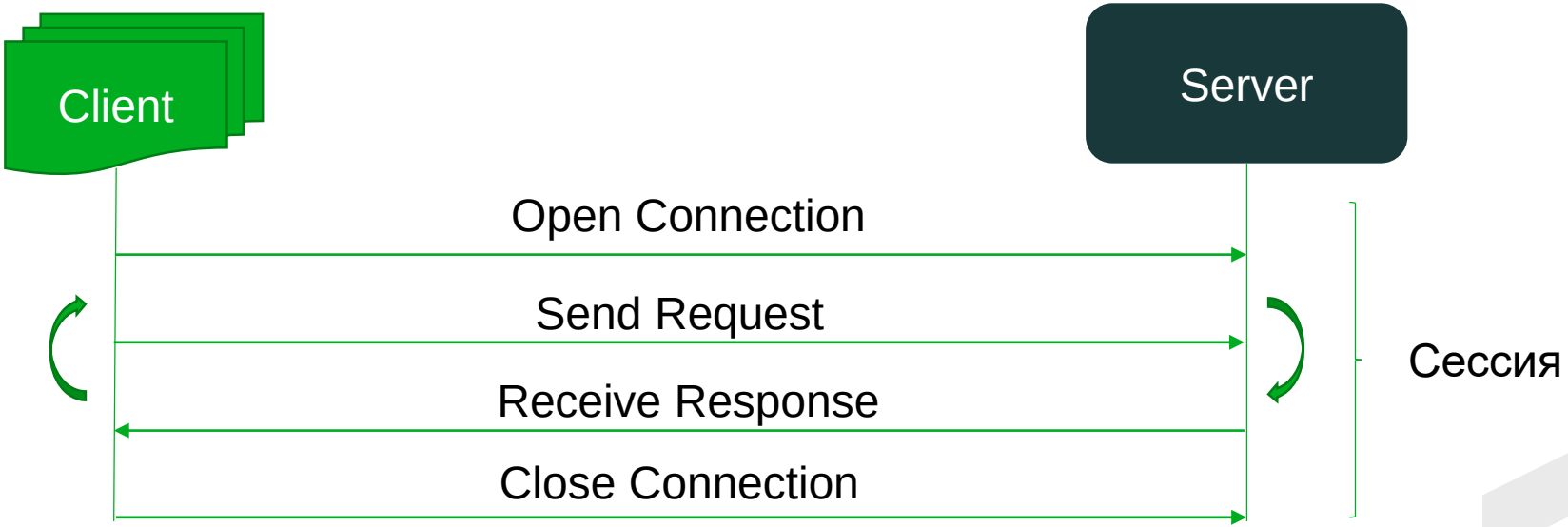


Классы для работы с сетью



TCP Networking

Transmission Control Protocol



Socket

это программный интерфейс для обмена данными между двумя узлами в сети.

Он позволяет программам на разных компьютерах (или даже внутри одного устройства) **устанавливать соединение и передавать данные**.

Клиент

```
var address = Inet4Address.getByName("localhost");
try (var socket = new Socket(address, 7777);
     var input = new DataInputStream(socket.getInputStream());
     var output = new DataOutputStream(socket.getOutputStream())) {

    output.writeUTF("Hello World");
    System.out.println("Response from server: " + input.readUTF());
}
```

Сервер

```
try (var serverSocket = new ServerSocket(7777);
     var socket = serverSocket.accept();
     var input = new DataInputStream(socket.getInputStream());
     var output = new DataOutputStream(socket.getOutputStream());) {

    System.out.println("Client request: " + input.readUTF());
    output.writeUTF("Hello from server!");
}
```

HTTP

HyperText Transfer Protocol

Каждое HTTP- сообщение, независимо от того, следует оно от клиента к серверу (request) или наоборот (response), состоит из 3 основных частей:

- Стартовая строка
- Заголовок (header)
- Тело сообщения (body)

Стартовая строка

определяет тип сообщения, в которой указываются следующие данные (general):

Request

- Методы (GET, POST, PUT, DELETE)
- Адрес (URL)
- Версия протокола (HTTP/1.1, HTTP/2)

GET
https://www.google.com/search?=java

Response

- Версия протокола (HTTP/1.1, HTTP/2)
- код состояния (1xx, 2xx,3xx …)
- текстовое пояснение

HTTP/2 200 OK

Методы HTTP запроса

Метод	CRUD	Entire Collection (e.g. /customers)	Specific Item (e.g. /customers/{id})
POST	Create	201 (Created), 'Location' header with link to /customers/{id} containing new ID.	404 (Not Found), 409 (Conflict) if resource already exists..
GET	Read	200 (OK), list of customers. Use pagination, sorting and filtering to navigate big lists.	200 (OK), single customer. 404 (Not Found), if ID not found or invalid.
PUT	Update/Replace	405 (Method Not Allowed), unless you want to update/replace every resource in the entire collection.	200 (OK) or 204 (No Content). 404 (Not Found), if ID not found or invalid.
PATCH	Update/Modify	405 (Method Not Allowed), unless you want to modify the collection itself.	200 (OK) or 204 (No Content). 404 (Not Found), if ID not found or invalid.
DELETE	Delete	405 (Method Not Allowed), unless you want to delete the whole collection—not often desirable.	200 (OK). 404 (Not Found), if ID not found or invalid.

https://www.restapitutorial.com/lessons/httpmethods.html

Заголовок (header)

характеризует тело сообщения и параметры его передачи в виде “header_name: header_value”

Иными словами – метаинформация HTTP сообщений

```
Authorization: Basic sNkjksYiosppassFfj20ZA==
Accept: text/html
Content-Type: application/json
Acccept-Language: en-US
Cookie: name1=vlaue1
```

Тело сообщения

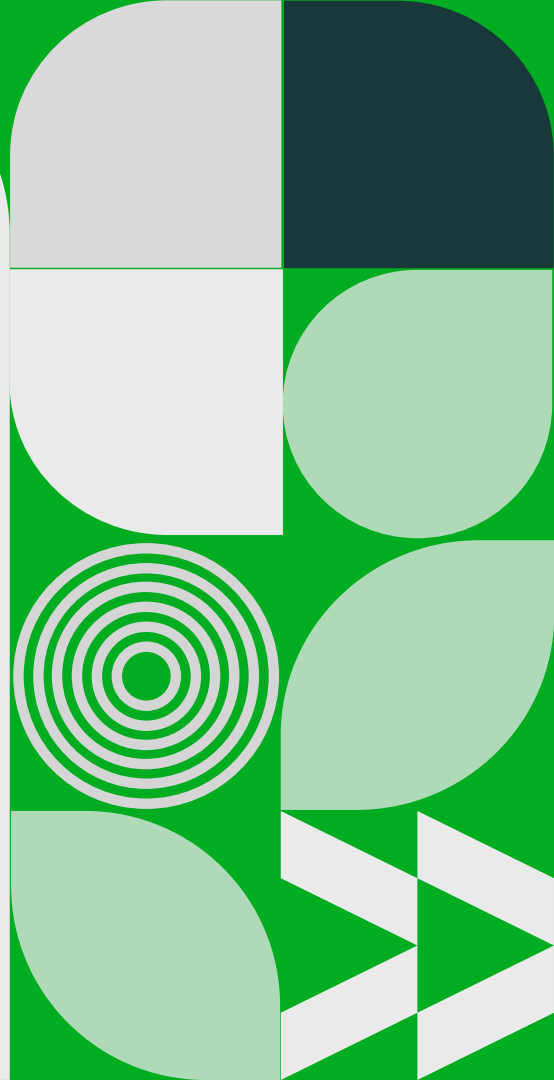
Body

Пересылаемые данные

Отделяются от заголовка пустой строкой.

Типы отправляемых данных могут быть любыми:

- HTML
- JSON
- XML
- Видео
- Картинки
- Файлы



HttpClient

java 11+

Позволяет удобно собирать все параметры
и отправлять запросы любых типов

```
var client = HttpClient.newHttpClient();
```

```
var request =
HttpRequest.newBuilder(URI.create("http://localhost:7777"))
      .headers("content-type", "application/json")
      .POST(ofFile(filePath))
      .build();
```

Java advanced
День 3

Основы Concurrency



nexign <academy>

Класс Thread и интерфейс Runnable

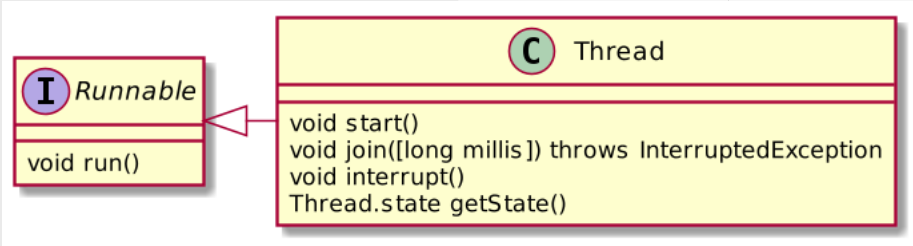
Когда использовать?

- Много CPU-ядер, вычислительная задача хорошо разделяется на подзадачи.
- Подзадачи блокируются на вводе/выводе, можно параллельно ждать других или делать что-то полезное.
- Нужно быстро отреагировать на запрос, а потом дать детальный ответ (пользовательский интерфейс).
- Многопользовательский сервис (каждый запрос в своём треде).

Интерфейс Runnable

- Нечто, что можно выполнить
- Содержит единственный метод void run();
- Может использоваться для описания действий, которые должны быть выполнены в отдельном потоке
- Вместе с тем, напрямую с потоками не связан и может быть использован и сам по себе

```
Runnable runnable = new Runnable() {
    @Override
    public void run() {
    // do some work
    }
};
Thread thread = new Thread(runnable);
```



Создание потока: Thread

- Выполняемый поток
- Реализует Runnable: class Thread implements Runnable
- Метод run() по умолчанию не делает ничего и должен быть переопределен в производном классе
- Для запуска потока на исполнение служит метод start()

```
Thread thread = new Thread () {
    @Override
    public void run () {
    // do some work
    }
}
```

В чем отличие?

MyThread ().run vs MyThread ().start()

- **join()**: ожидает завершение потока
- **run()**: определяет точку входа в поток
- **sleep()**: приостанавливает поток на заданное количество миллисекунд
- **start()**: запускает поток, вызывая его метод run()
- **getName()**: возвращает имя потока
- **setName(String name)**: устанавливает имя потока
- **getPriority()**: возвращает приоритет потока
- **setPriority(int proirity)**: устанавливает приоритет потока.. В этот метод в качестве параметра передается числовое значение приоритета - от 1 до 10. По умолчанию главному потоку выставляется средний приоритет - 5.
- **isAlive()**: возвращает true, если поток активен
- **isInterrupted()**: возвращает true, если поток был прерван

Создание и запуск

```
class CalcSquare extends Thread {
    final int argument;
    int result;
    CalcSquare(int argument) {
        this.argument = argument;
    }
    @Override
    public void run() {
        //«сложные» вычисления
        result = argument * argument;
    }
}
```

```
CalcSquare t1 = new CalcSquare(2);
CalcSquare t2 = new CalcSquare(3);
t1.start();
t2.start();
t1.join();
t2.join();
System.out.printf("%d, %d%n", t1.result, t2.result);
//otput: 4, 9
```

Стандартные потоки

- Главный поток
  - Создаётся при запуске программы
  - Выполняет функцию main. Может создавать другие потоки
  - Они, в свою очередь, могут создавать ещё потоки
- Графические библиотеки
- Поток обработки сообщений (обычно создаётся внутри библиотеки)
- Дополнительные потоки сборщика мусора (Garbage Collector Threads)

Жизненный цикл потока

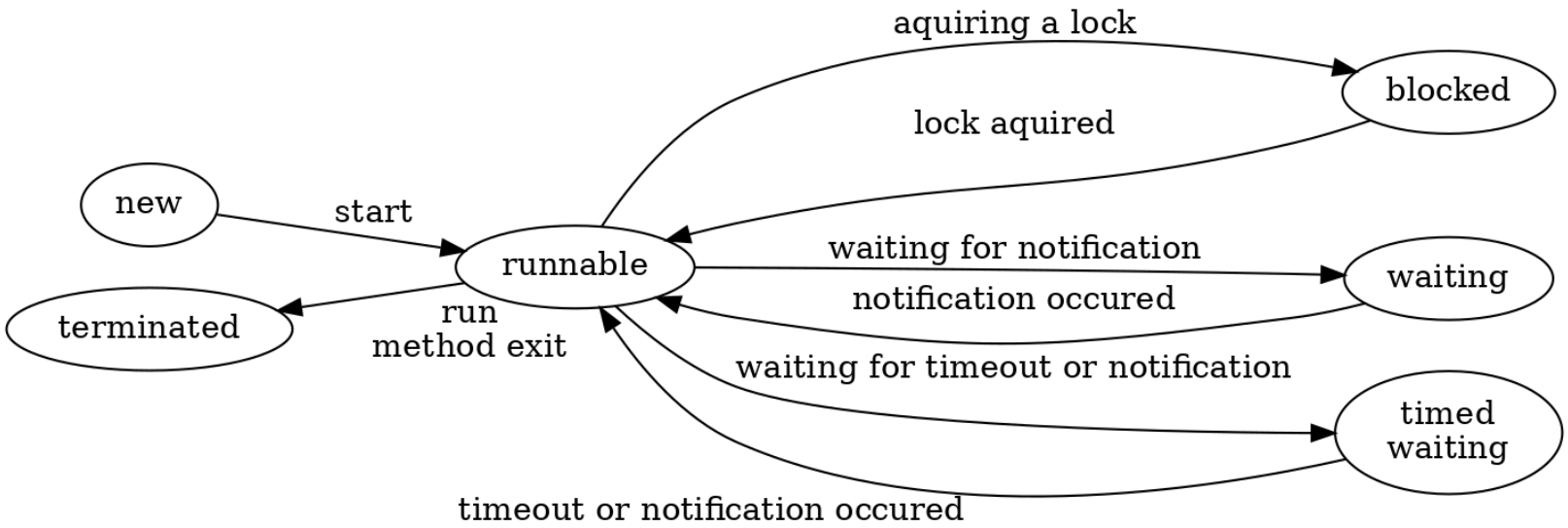
1. Создание объекта Thread
2. Запуск

   thread.start()
3. Работа

   выполняется метод **run(), thread.isAlive() == true**
4. Завершение

   метод **run()** закончился или бросил исключение
5. Завершенный поток нельзя перезапустить

Состояния треда



Прерывание потока

- thread.interrupt()
- Если поток находится в ожидании (sleep, join, wait), то ожидание прерывается исключением InterruptedException
- Иначе у потока просто устанавливается флаг interrupted
    - флаг проверяется методами interrupted() и isInterrupted()
    - проверять флаг (isInterrupted) и завершать поток надо самостоятельно
- thread.join()

Планировщик потоков

Планировщик потоков
– это часть JVM, которая решает какой поток должен выполнится в каждый конкретный момент времени и какой поток нужно приостановить.

Действия при переключении контекста

1. Сохранение контекста потока, закончившего выполнение.
2. Помещение этого потока в конец очереди, соответствующей его приоритету.
3. Загружает контекст потока из очереди готовых к выполнению с наибольшим приоритетом.
4. Удаляет загруженный поток из очереди и начинает его выполнять.

Правила, определяющие когда должно происходить переключение контекста

- Закончился выделенный планировщиком квант времени.
- Поток может добровольно уступить управление. Для этого достаточно явно уступить очередь на исполнение, приостановить или блокировать поток на время ожидания ввода вывода.
- Один поток исполнения может быть вытеснен другим, более приоритетным потоком.

Гарантии Java

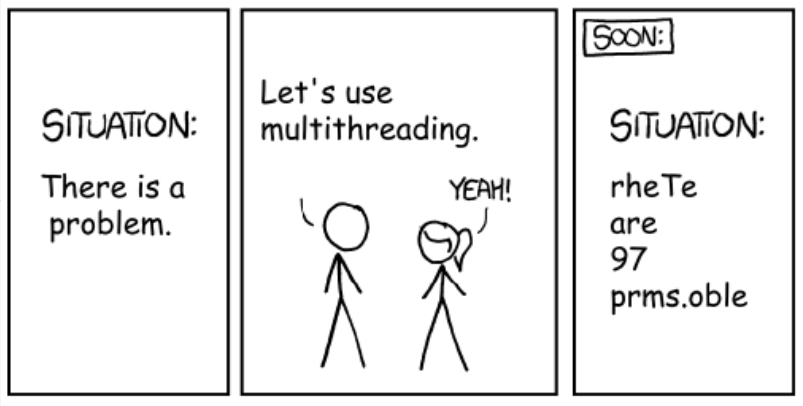
“When it comes to threads, very little is guaranteed.”

- Java НЕ гарантирует, что потоки будут выполнены в том порядке в котором они были запущены.
- Нет гарантии, что если поток начал свое выполнение, то он выполнит свою работу не прерываясь.
- При каждом новом запуске программы, результат может быть разным.
- Гарантируется, что каждый поток начнет свою работу и будет выполнять пока не закончит.
- Внутри одного потока действия выполняются в предсказуемом порядке.

Разделенный ресурс

Проблемы shared state

- Race condition
- Stale values
- Reordering



Reordering

Компилятор может переставлять порядок выполнения команд

```
class PossibleReordering {
    static int x = 0, y = 0, a = 0, b = 0;
    public static void main(String... args)
            throws InterruptedException {
        //другой способ запуска треда
        Thread one = new Thread(() -> {
            a = 1; x = b;
        });
        Thread two = new Thread(() -> {
            b = 1; y = a;
        });
        one.start(); two.start();
        one.join();  two.join();
        System.out.printf("%d,%d", x, y);
        //??,??
    }
}
```

Race Condition

```
class DumbCounter {
    int count;
    void increment(){
        count++;
    }
}

DumbCounter c1 = new DumbCounter();
IntStream.range(0, 1000000).forEach(i->c1.increment());

DumbCounter c2 = new DumbCounter();
IntStream.range(0, 1000000).parallel().forEach(i->c2.increment());

System.out.printf("%d, %d%n", c1.count, c2.count);

//1000000,??????
```

State Values

```
class DumbWayToFallAsleep implements Runnable
{
    private boolean asleep;

    public void setAsleep(boolean asleep){
        this.asleep = asleep;
    }

    @Override
    public void run() {
        while (!asleep){
            //countSomeSheep
            //УСНЁМ ЛИ МЫ?
        }
    }
}
```

Java Memory Model

- Часть спецификации языка Java
- Описывает взаимодействие приложения с памятью
- Не зависит от реализации JVM, операционной системы и железа
- отвечает на вопрос: «при каких условиях поток, читающий переменную увидит значение»

JMM Rules

Правило № 1: однопоточные программы исполняются псевдопоследовательно.

Это значит: в реальности процессор может выполнять несколько операций за такт, заодно изменив их порядок, однако все зависимости по данным остаются, так что поведение не отличается от последовательного.

Правило № 2: нет невесть откуда взявшихся значений.

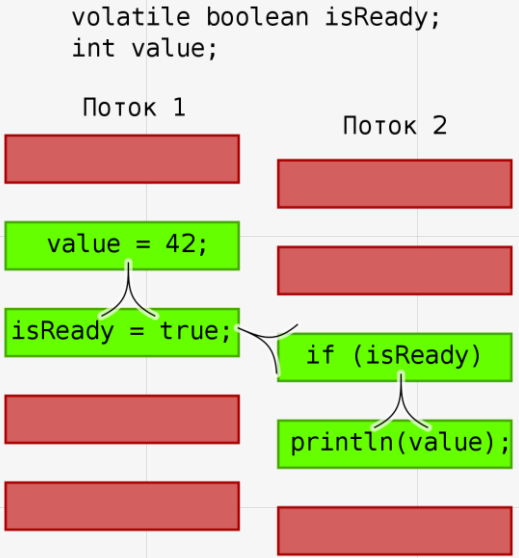
Чтение любой переменной (кроме не-volatile long и double, для которых это правило может не выполняться) выдаст либо значение по умолчанию (ноль), либо что-то, записанное туда другой командой.

И правило № 3: остальные события выполняются по порядку, если связаны отношением строгого частичного порядка «выполняется прежде» (англ. happens before).

Happens before

- JMM определяет частичный порядок на всех действиях в Java-программе, называемый *happens-before*.
- Отношение *happens-before* транзитивно: $A \prec B \wedge B \prec C \Rightarrow A \prec C$
- Чтобы действие B гарантированно видело результат работы действия A, достаточно, чтобы $A \prec B$.

- Вызов thread.start() hb первое действие в потоке thread
- Последнее действие в потоке thread hb thread.join()
- Инициализация объекта по умолчанию hb любое другое действие



volatile

- Переменные классов могут быть определены с ключевым словом volatile.
- **Запись в volatile-переменную *happens-before* чтения из этой переменной в другом потоке.**
- Это автоматом делает видимыми изменения в других переменных. Полагаться на это не рекомендуется: это работает, но делает код хрупким. В процессе рефакторинга можно поменять порядок доступа к переменным и тем самым незаметно поломать программу.

Речь идёт непосредственно о записи, а не о записи членов/элементов массива

```
volatile int[] x;
x = new int[10]; // volatile write
x[0] = 1; // volatile read, plain write
```

Solve stale value

```
class NotSoDumbWayToFallAsleep implements Runnable {
    private volatile boolean asleep;

    public void setAsleep(boolean asleep){
        this.asleep = asleep;
    }

    @Override
    public void run() {
        while (!asleep){
            //countSomeSheep
            //...
        }
    }
}
```

Доступ к разделяемому ресурсу

Блокировка - взаимное исключение (пока один поток что-то делает, другие не могут ему помешать)

- Поток ожидает уведомление от других

Неблокирующий доступ

- Lock-free: гарантируется общий прогресс
- Wait-free: гарантируется прогресс в каждом потоке исполнения



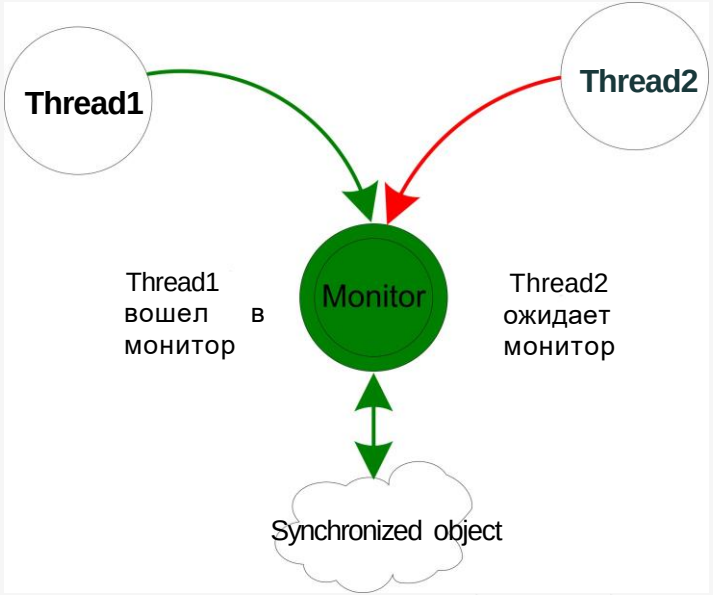


Синхронизация

- **Синхронизация по ресурсам (управление ресурсами)**
  - Объекты совместно используют ресурсы данных
  - Ресурсы сами следят за корректностью работы
  - Java: synchronized + volatile
- **Синхронизация по событиям (управление временем)**
  - Для совместной работы потоки уступают процессорное время друг другу и/или уведомляют друг друга о возможности продолжения работы
  - Java: Object.wait(), notify(), notifyAll()

Монитор


Придется
подождать
Занято
Поток 1


Thread1
Thread2
Thread1
вошел в
монитор
Monitor
Thread2
ожидает
монитор
Synchronized object

synchronized

- **Синхронизация блоков** — по монитору указанного объекта
- **Синхронизация методов** — по монитору текущего объекта (this)
- **Синхронизация статических методов** — по монитору класса

- Между синхронизациями по одному объекту установлен полный порядок (total order)
- Завершение синхронизации (monitorexit) hb начало последующей синхронизации по тому же объекту (monitorenter)

Синхронизованный метод

```
public synchronized void doSomething () {
    // ...
}
```

Синхронизованный блок внутри метода

```
public void doSomething () {
    synchronized ( obj ) {
        // ...
    }
}
```

Non-sync

Class Client

```
Deposit deposit = new Deposit( 300 );
Client client = new Client( deposit, -100 );
Client bank = new Client( deposit, 1000 );

bank.start();
client.start();

try {
    bank.join();
    client.join();
    int balance = deposit.getBalance();
    System.out.println( "Balance at termination: " + balance );
}
catch( InterruptedException e ) {
    System.out.println( "Unexpected");
```



Non-sync

Class Client

```
    @Override
    public void run() {
        try {
            int balance = deposit.getBalance();
            System.out.println("Client: balance before transaction: " + balance);
            balance += change;
            deposit.setBalance(balance);
            System.out.println("Client: balance: " + balance);
        } catch( InterruptedException e ) {
            System.out.println( "Interrupting client thread");
        }
    }
}
```

Sync

```
    public void run() {
        try {
            synchronized(deposit) {
                int balance = deposit.getBalance();
                System.out.println( "Client: balance before transaction: " + balance );
                balance += change;
                deposit.setBalance(balance);
                System.out.println( "Client: balance: " + balance );
            }
        }
        catch( InterruptedException e ) {
            System.out.println( "Interrupting client thread");
        }
    }
}
```

Immutable state

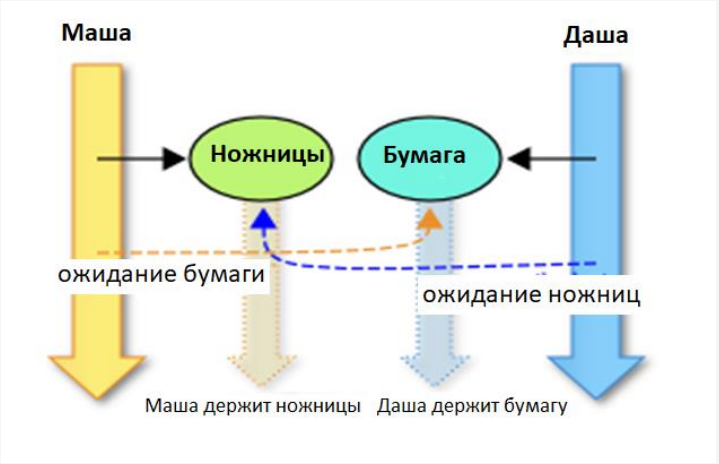
> Если поток увидел ссылку на объект, и она не утекала из конструктора, то он гарантированно увидит все final-поля, записанные в конструкторе
>
> **Неизменяемые объекты – друзья многопоточности!**



Deadlock

Как происходит блокировка?

- Два потока работают с общими ресурсами.
- Поток 1 захватывает Ресурс 1 и начинает операции с ним.
- Поток 2 последовательно захватывает Ресурс 2 и Ресурс 1.
- Поток 2 не получает доступа к Ресурсу 1 и в ступоре ждёт, когда тот освободится.
- Поток 1 не завершил работу с Ресурсом 1, но пытается захватить Ресурс 2 и тоже впадает в ступор.



Ожидание и уведомления

Допустимо только внутри synchronized

- **void wait()** – уступить монитор и перейти в режим ожидания
- **void notify()** – пробудить один из потоков, вызвавший wait() на данном объекте
- **void notifyAll()** –пробудить все потоки, вызвавшие wait() на данном объекте

Notify

```
public synchronized int getMoney(int requested) throws InterruptedException {
    assert requested > 0: "Можно запросить только положительную сумму";
    // Задержка имитирует возможную трудоемкость процедуры записи
    Thread.sleep(10);
    if (requested > balance) {
        wait(5000);
        if (requested > balance) {
            int oldBalance = balance;
            balance = 0;
            return oldBalance;}}
    balance -= requested;
    return requested;}
public synchronized int putMoney(int sum) throws InterruptedException {
    assert sum > 0 : "Можно добавить только положительную сумму";
    // Задержка имитирует возможную трудоемкость процедуры записи
    Thread.sleep(10);
    balance += sum;
    notifyAll();
    return balance;}
```

java.util.concurrent.*

ReentrantLock

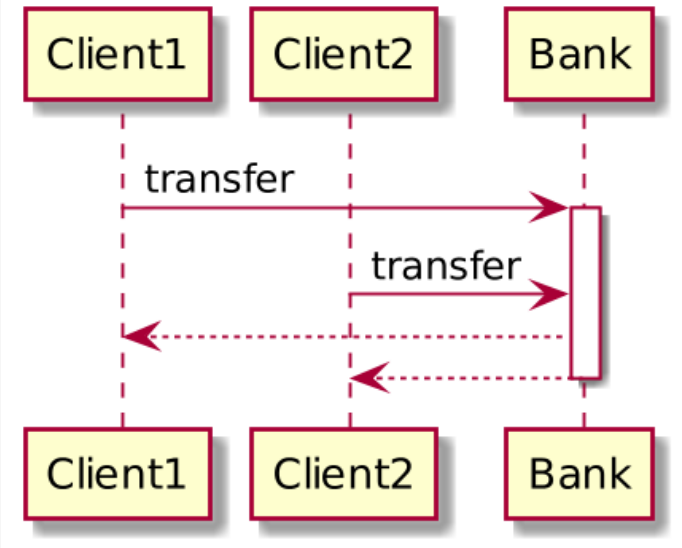
- Класс java.util.concurrent.locks.ReentrantLock
- Обеспечивает взаимное исключение потоков, аналогичное synchronized-блокам
- Операции:
  - lock()
  - unlock()

```
Lock lock = new ReentrantLock();
lock.lock();
try {
    doSomething();
} finally {
    lock.unlock();
}
```

Не синхронизованное выполнение

```
void dumbMoneyTransfer(
        int from, int to, int amount){

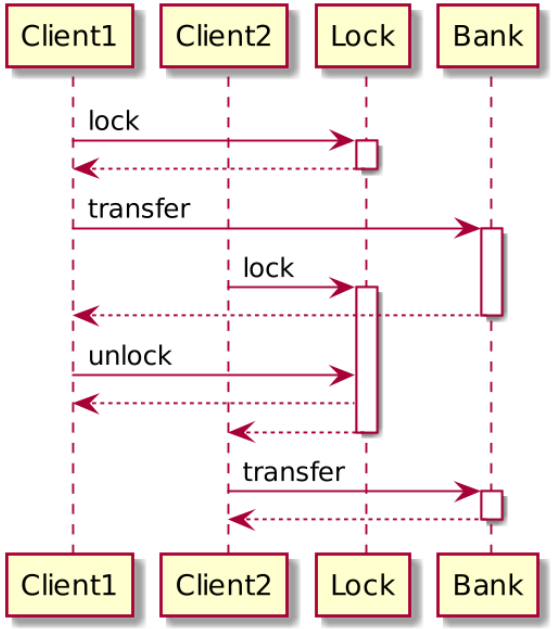
    accounts[from]-=amount;
    accounts[to]+=amount;
}
```



Синхронизованное выполнение

```
private ReentrantLock bankLock = new ReentrantLock();

void moneyTransfer(int from, int to, int amount) {
    bankLock.lock();
    try {
        accounts[from]-=amount;
        accounts[to]+=amount;
    } finally {
        bankLock.unlock();
    }
}
```



Condition

- Класс java.util.concurrent.locks.Condition
- Аналог wait/notify
- Привязан к Lock’у
- У одного Lock’а может быть много Condition’ов

```
Lock lock = new ReentrantLock();
Condition condition = lock.newCondition ();
lock.lock();
try {
    while (!conditionSatisfied()) {
        condition.await ();
    }
} finally {lock.unlock(); }
    // somewhere else in our program
lock.lock ();
try {
    condition.signal();
} finally { lock.unlock();}
```

Потокобезопасные структуры

Блокировка (через synchronized или ReentrantLock) решает вопрос координации действий разных тредов с переменной.
Но если много тредов конкурируют за блокировку (high lock contention), затраты ресурсов на координацию тредов становятся значительными.

Альтернативой являются *неблокирующие алгоритмы*, использующие поддержку специальных атомарных машинных инструкций (compare-and-swap).
В Java-библиотеке доступны *классы атомарных переменных* и *потокобезопасные коллекции*, реализованные в том числе на неблокирующих алгоритмах.

java.util.concurrent

- Атомарные переменные
- Неблокирующие коллекции
- Блокирующие коллекции
- Примитивы синхронизации
- Пулы потоков
- CompletableFuture

Atomics

java.util.concurrent.atomic

- AtomicBoolean
- AtomicInteger
- AtomicLong
- AtomicReference<V>

Операции

- getAndSet(newValue)
- compareAndSet(expect, update)
- incrementAndGet() /decrementAndGet()
- getAndIncrement() /getAndDecrement()
- getAndAdd(delta)
- addAndGet(delta)
- getAndUpdate(updateFunction)
- updateAndGet(updateFunction)

Compare and Set

- (expect, update) -> result
- Базовый примитив для многих lock-free алгоритмов

```
private volatile boolean flag = false;
void doOnce(Runnable action) {
    if (!flag) {
        flag = true;
        action.run();
    }
}
```



```
private final AtomicBoolean flag =
            new AtomicBoolean(false);
void doOnce(Runnable action) {
    if (flag.compareAndSet(false, true)) {
        action.run();
    }
}
```



Пример реализации

```
public final int incrementAndGet() {
    for (;;) {
        int current = get();
        int next = current + 1;
        if (compareAndSet(current, next))
            return next;
    }
}
```

Примитивы синхронизации

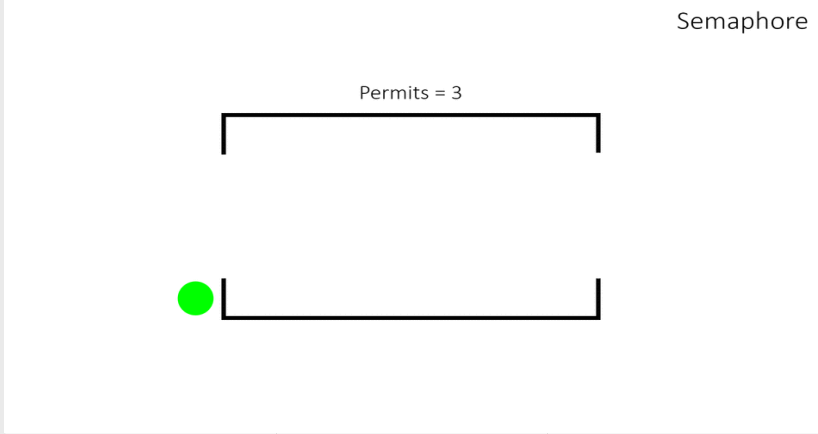
Синхронизаторы

вспомогательные утилиты для синхронизации потоков, которые дают возможность разработчику регулировать и/или ограничивать работу потоков и предоставляют более высокий уровень абстракции, чем основные примитивы языка (мониторы).

- Semaphore
- CountDownLatch
- CyclicBarrier
- Exchanger
- Phaser

Semaphore

- Класс java.util.concurrent.Semaphore
- Ограничивает одновременный доступ к ресурсу
- В отличие от synchronized-блока, одновременно могут работать несколько потоков (но не более заданного N)
- **Операции:**
  - void acquire()
  - void release()

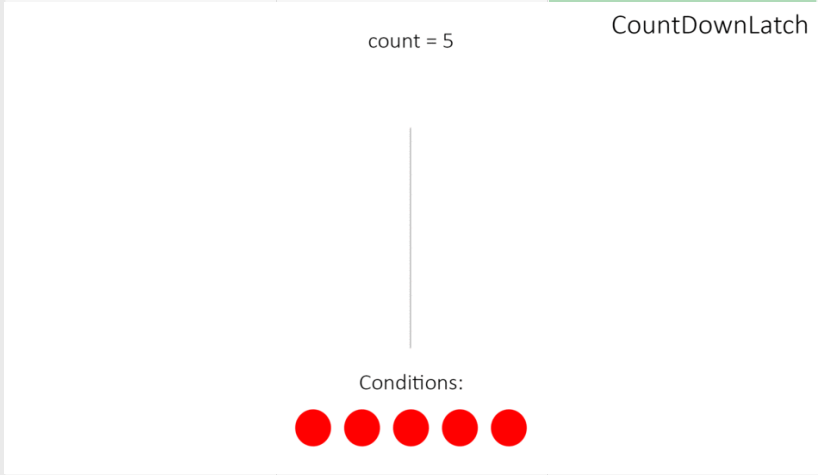


Semaphore

```
Semaphore semaphore = new Semaphore(10);
semaphore.acquire();
try {
    // up to 10 threads may
    // execute this code concurrently
} finally {
    semaphore.release();
}
```

CountDownLatch

- Класс java.util.concurrent.CountDownLatch
- Обеспечивает точку синхронизации между N потоками (несколько потоков могут дожидаться друг друга и потом стартовать одновременно)
- **Операции:**
  - void await()
  - void countDown()



CountDownLatch

```
CountDownLatch latch = new CountDownLatch(10);
    // this call blocks until latch.countDown ()
    // is called at least 10 times
latch.await();
```

CyclicBarrier

- Класс java.util.concurrent.CyclicBarrier
- Вариант CountDownLatch, допускающий повторное ожидание



Executors

- Тред — дорогой ресурс, поэтому мы хотим:
  - ограничивать количество наших тредов, чтобы не устроить Out of Memory,
  - переиспользовать имеющиеся треды, подавая им новые задачи после завершения старых,
- но если какой-то тред «вылетел» — автоматически создавать новый.
- В стандартной библиотеке для этого есть Thread Pools, не надо ничего делать самостоятельно.
- Никто не использует Thread API напрямую

ExecutorService

- Класс java.util.concurrent.Executors
  - **ExecutorService newSingleThreadExecutor() –** пул одного потока
  - **ExecutorService newFixedThreadPool(int nThreads**) - фиксированный размер пула
  - **ExecutorService newCachedThreadPool()** - пул растёт по необходимости, держит неактивный тред 60 секунд

Future

> Абстракция результата "in progress", который можно ждать, а можно и отменить

```
public interface Future<V> {
    V get() throws InterruptedException, ExecutionException;
    V get(long timeout, TimeUnit unit)
            throws InterruptedException, ExecutionException, TimeoutException;
    boolean cancel(boolean mayInterruptIfRunning);
    boolean isDone();
    boolean isCancelled();
}
```

Обработка задач

Запускаем и ждём, пока все выполнятся

```
<T> List<Future<T>> invokeAll(
            Collection<? extends Callable<T>> tasks)
    throws InterruptedException
```

Запускаем, возвращаем первый успешный результат, отменяем остальные

```
<T> T invokeAny(Collection<? extends Callable<T>> tasks)
            throws InterruptedException, ExecutionException;
```

CompletableFuture

- Появились в Java 8, расширяют Future.
- Позволяют явно задать результат (отсюда 'Completable') и собрать цепочку асинхронных вычислений.

```
CompletableFuture<Integer> f = new CompletableFuture<>();
executor.execute(() -> { int n = workHard(arg);
    f.complete(n);});

executor.execute(() -> {int n = workSmart(arg);
    f.complete(n);});

executor.execute(() -> {Throwable t = ...;
    f.completeExceptionally(t); });
```

Пример

```
static BigInteger factorial(int i) {
    BigInteger res = BigInteger.ONE;
    while(i > 1) {
        res = res.multiply(BigInteger.valueOf(i));
        i--;
    }
    return res;
}
```

```
static BigInteger combinations(int n, int k) {
    CompletableFuture<BigInteger> factN = supplyAsync(() -> factorial(n));
    CompletableFuture<BigInteger> factK = supplyAsync(() -> factorial(k));
    CompletableFuture<BigInteger> factNminusK = supplyAsync(() -> factorial(n-k));
    return factN.thenCombine(factK, BigInteger::divide)
            .thenCombine(factNminusK, BigInteger::divide).join();
}
```

Concurrent collections

Неблокирующие коллекции

- ConcurrentLinkedQueue (LinkedList)
- ConcurrentLinkedDeque (LinkedList/ArrayDeque)
- CopyOnWriteArrayList (ArrayList)
- CopyOnWriteArraySet
- ConcurrentHashMap
  (+ newKeySet()) (HashMap/HashSet)
- ConcurrentSkipListMap (TreeMap)
- ConcurrentSkipListSet (TreeSet)

- Простые операции атомарны
- Пакетные операции (addAll, removeAll) могут быть неатомарны!
- Как правило, длина не хранится (isEmpty()!)
- Не кидает ConcurrentModificationException
- могут быть видны или не видны

CopyOnWriteArrayList и CopyOnWriteArraySet

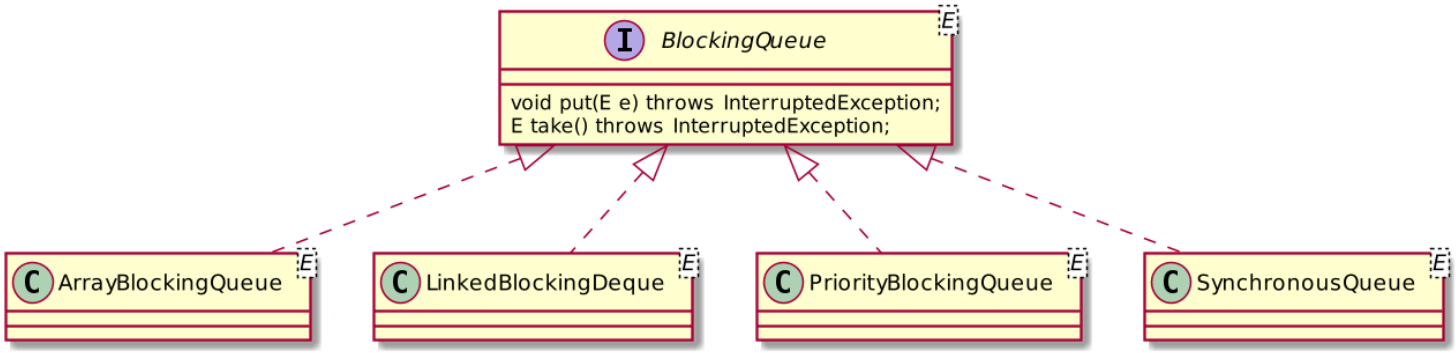
- Содержимое хранится в массиве
- Модифицирующие операции синхронизированы
- Операции на чтение и iterator не синхронизированы
- Любое изменение копирует массив
- Итератор, forEach обходят снимок коллекции на момент начала итерации (итератор не может модифицировать).
- Сортировать можно с Java 8
- CopyOnWriteArraySet – делегат к CopyOnWriteArrayList

Concurrent collections

Блокирующие коллекции

- SynchronousQueue (без памяти, одна вставка = одно удаление)
- ArrayBlockingQueue (фиксированный размер)
- LinkedBlockingDeque (размер может быть не фиксирован)
- PriorityBlockingQueue (PriorityQueue x ArrayBlockingQueue)

BlockingQueue



- Могут быть ограничены по размеру (capacity constrained).
- Методы put() и take() ждут, пока не появится возможность положить или взять элемент.
- PriorityBlockingQueue не лимитируется по capacity.
- SynchronousQueue не имеет capacity вовсе, передаёт элементы обрабатывающим тредам напрямую.

Пример

```
BlockingQueue<String> syncQueue = new SynchronousQueue<>();
Runnable producer = () -> {
    int i=0;
    while (true) try {
        Thread.sleep(ThreadLocalRandom.current().nextInt(500, 1000));
        String value = String.valueOf(i++);
        syncQueue.put(value);
        System.out.println("Added: "+value);
    } catch (InterruptedException ignored) { } };
Runnable consumer = () -> {
    while (true) try {
        Thread.sleep(ThreadLocalRandom.current().nextInt(500, 2000));
        System.err.println("Read: " + syncQueue.take());
    } catch (InterruptedException ignored) { } };
new Thread(producer).start();
new Thread(consumer).start();
```

Многопоточность в сторонних зависимостях

Connection pool

набор (очередь) соединений, которые открываются один раз и далее переиспользуются по мере необходимости каждый раз возвращаясь в пул обратно.

«ручная реализация»

```
private static void initConnectionPool() {
    pool = new ArrayBlockingQueue<>(DEFAULT_POOL_SIZE);
    for (int i = 0; i < DEFAULT_POOL_SIZE; i++) {
        Connection open = open();
        Connection proxyInstance =
                (Connection) Proxy.newProxyInstance(
                        ConnectionManager.class.getClassLoader(),
                        new Class[]{Connection.class},
                        (proxy, method, args) -> method.getName().equals("close")
                                ? pool.add((Connection) proxy)
                                : method.invoke(open, args));
        pool.add(proxyInstance);

    }
```

сторонняя зависимость

HikariCP

Спасибо!

